

SME WEBSITE SECURITY AUDIT REPORT

A Controlled Lab Assessment Using DVWA as the Simulated SME Web Application

Prepared by: Akinwumi Oni

Fellow ID: FE/26/717977653

Cybersecurity NextGen Cohort — Capstone Project

Project: SME Website Security Audit

Assessment Date: 6 August 2026

CONFIDENTIAL — LAB / TRAINING ENVIRONMENT ONLY

This assessment was performed exclusively against a self-hosted, isolated virtual lab (DVWA on Ubuntu) and was never conducted against a live production system or any host without explicit authorization.

Table of Contents

1. Executive Summary	3
2. Scope & Rules of Engagement	3
2.1 In-Scope Target.....	3
2.2 Out of Scope.....	3
2.3 Authorization.....	4
3. Methodology	4
4. Lab Environment & Architecture	4
5. Tools Used	5
6. Reconnaissance Findings	5
7. Automated Vulnerability Scan Findings (OWASP ZAP)	6
8. Manual Exploitation Findings.....	8
8.1 SQL Injection — Critical (F-01)	8
8.2 Stored Cross-Site Scripting (XSS) — High (F-02).....	10
9. Network Traffic Analysis (Wireshark)	11
10. Consolidated Risk Summary	13
11. Remediation & Hardening Recommendations	13
11.1 SQL Injection (F-01) — Remediated.....	13
11.2 Stored XSS (F-02) — Recommended Fix.....	14
11.3 Missing Transport Encryption (F-03) — Remediated	14
11.4 Security Header & Cookie Hardening	15
11.5 Information Disclosure (F-07, F-11).....	15
12. Verification of Fixes	15
13. Conclusion	16
Appendix A — Evidence Index.....	16
Appendix B — Disclaimer.....	17

1. Executive Summary

This report documents a controlled security audit performed as the capstone project for the Cybersecurity NextGen Cohort. The engagement simulates a small-and-medium enterprise (SME) website assessment using the Damn Vulnerable Web Application (DVWA) as a stand-in for a real SME site, hosted on a self-managed Ubuntu server and assessed from a Kali Linux attacker workstation, both running as isolated virtual machines on VMware with host-only/NAT networking.

The objective was to demonstrate a full, safe, lab-based security review lifecycle: reconnaissance, automated vulnerability scanning, manual exploitation of high-impact findings, network traffic analysis, and remediation — with each stage evidenced by screenshots and verified after fixes were applied.

Eleven distinct findings were identified and risk-rated, ranging from Critical to Informational. Two of the most severe issues — a Critical SQL Injection vulnerability and unencrypted HTTP traffic — were actively exploited or confirmed and were subsequently remediated on the host: the vulnerable query was rewritten to use a parameterized PDO prepared statement, and the site was reconfigured to serve over HTTPS using a self-signed certificate. A Stored XSS vulnerability was also confirmed exploitable and is documented with a specific, low-effort remediation path.

Key Statistics

Metric	Result
Total findings identified	11 (1 Critical, 2 High, 3 Medium, 4 Low, 1 Informational)
Findings actively exploited (proof of concept)	2 (SQL Injection, Stored XSS)
Findings remediated and verified in this engagement	2 (SQL Injection, Missing Transport Encryption)
Automated scanner used	OWASP ZAP 2.17.0 (Automated Scan + manual alert review)
Total ZAP alerts raised against target	13 (see Section 7)
Host services exposed at scan time	TCP/80 (HTTP, Apache 2.4.63 on Ubuntu); TCP/22 closed

2. Scope & Rules of Engagement

2.1 In-Scope Target

- Target application: Damn Vulnerable Web Application (DVWA), acting as the simulated SME website.
- Target host: Ubuntu Server VM, IP address 192.168.126.129, running Apache 2.4.63 and MariaDB 11.4.7.
- Vulnerability categories in scope: SQL Injection, Cross-Site Scripting (Reflected/Stored), security misconfigurations, session/cookie handling, transport encryption, and information disclosure.

2.2 Out of Scope

- Any host, IP address, or service not explicitly listed above.
- Denial-of-service testing, physical security testing, and social engineering.
- Any production or third-party system. All testing was confined to an isolated, host-only VMware lab network with no internet-facing exposure.

2.3 Authorization

This assessment was self-authorized as a training exercise against infrastructure owned and operated solely by the assessor (Akinwumi Oni), deployed specifically for this capstone. DVWA itself is intentionally vulnerable software distributed for security training and explicitly warns against exposing it outside a controlled lab — a warning that was respected throughout this engagement.

3. Methodology

The assessment followed a standard black-box-to-grey-box web application testing methodology, broadly aligned with the OWASP Testing Guide and the PTES framework, executed in five phases:

- Phase 1 — Reconnaissance: Host discovery and service/version enumeration with Nmap; passive traffic capture with Wireshark.
- Phase 2 — Automated Scanning: Spidering and active scanning of the target with OWASP ZAP to enumerate attack surface and flag misconfigurations.
- Phase 3 — Manual Exploitation: Targeted, hands-on testing of the highest-value findings (SQL Injection, Stored XSS) to confirm real-world exploitability beyond scanner output.
- Phase 4 — Analysis & Risk Rating: Consolidating findings, assigning risk based on likelihood and impact, and mapping each to a remediation.
- Phase 5 — Remediation & Verification: Applying fixes on the host and re-testing to confirm the vulnerability was closed.

4. Lab Environment & Architecture

Component	Details
Attacker machine	Kali Linux (VMware VM) — Firefox ESR, OWASP ZAP 2.17.0, Nmap 7.98, Wireshark 4.6.4
Target / SME host	Ubuntu (VMware VM), hostname ima-obong-VMware-Virtual-Platform — Apache 2.4.63, MariaDB 11.4.7, PHP 8.4
Target application	DVWA (cloned from github.com/digininja/DVWA), security level: Low
Network	Isolated VMware host-only/NAT segment, 192.168.126.0/24
Target IP	192.168.126.129

The Ubuntu host was built from scratch for this project: LAMP stack installed via apt, a dedicated dvwa_user MariaDB account created and scoped to a dedicated dvwa_db database (avoiding use of the MySQL root account), and DVWA deployed via git clone with its config.inc.php pointed at the local database. This mirrors how a real SME site is typically provisioned, which is what makes the findings representative of real-world risk rather than an artificial default install.

```

Aug 6 14:49
ima-obong@ima-obong-VMware-Virtual-Platform: ~
ima-obong@ima-obong-VMware-Virtual-Platform:~$ sudo apt update && sudo apt install -y apache2 mariadb-server php php-mysql php-gd git
[sudo] password for ima-obong:
Sorry, try again.
[sudo] password for ima-obong:
Hit:1 http://security.ubuntu.com/ubuntu plucky-security InRelease
Hit:2 http://ng.archive.ubuntu.com/ubuntu plucky InRelease
Hit:3 http://ng.archive.ubuntu.com/ubuntu plucky-updates InRelease
Hit:4 http://ng.archive.ubuntu.com/ubuntu plucky-backports InRelease
All packages are up to date.
Note, selecting 'php8.4-mysql' instead of 'php-mysql'
apache2 is already the newest version (2.4.63-1ubuntu1.1).
mariadb-server is already the newest version (1:11.4.7-0ubuntu0.25.04.1).
php is already the newest version (2:8.4+96ubuntu1).
php8.4-mysql is already the newest version (8.4.5-1ubuntu1.2).
php-gd is already the newest version (2:8.4+96ubuntu1).
git is already the newest version (1:2.48.1-0ubuntu1.1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
ima-obong@ima-obong-VMware-Virtual-Platform:~$ sudo mariadb -u root
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 31
Server version: 11.4.7-MariaDB-0ubuntu0.25.04.1 Ubuntu 25.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Support MariaDB developers by giving a star at https://github.com/MariaDB/server
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>

```

Figure 1 — LAMP stack installation and dedicated, least-privilege database account creation on the Ubuntu SME host.

5. Tools Used

Tool	Purpose
Nmap 7.98	Port and service/version discovery against the target host
OWASP ZAP 2.17.0	Automated spidering, active scanning, and manual alert triage
Wireshark 4.6.4	Passive capture and inspection of network traffic to/from the target
Kali Linux browser (Firefox)	Manual exploitation and verification of SQL Injection and Stored XSS
nano / bash / MariaDB CLI	Source-code and database-level remediation on the Ubuntu host
OpenSSL / a2enmod	Generating a self-signed certificate and enabling HTTPS on Apache

6. Reconnaissance Findings

An Nmap service/version scan was run against the target host to establish the external attack surface before any application-layer testing began:

```
nmap -sV -sC -p 80,22 192.168.126.129 -oN nmap_sme_scan.txt
```

Port	State	Service	Version / Notes
22/tcp	closed	ssh	No SSH exposure at scan time
80/tcp	open	http	Apache httpd 2.4.63 (Ubuntu); title 'Apache2 Ubuntu Default Page: It works'

Observation: the Apache banner discloses the exact server version and OS distribution (Apache/2.4.63 (Ubuntu)), which is useful reconnaissance for an attacker fingerprinting the stack for known CVEs. This corresponds to finding F-07 (see Section 10).

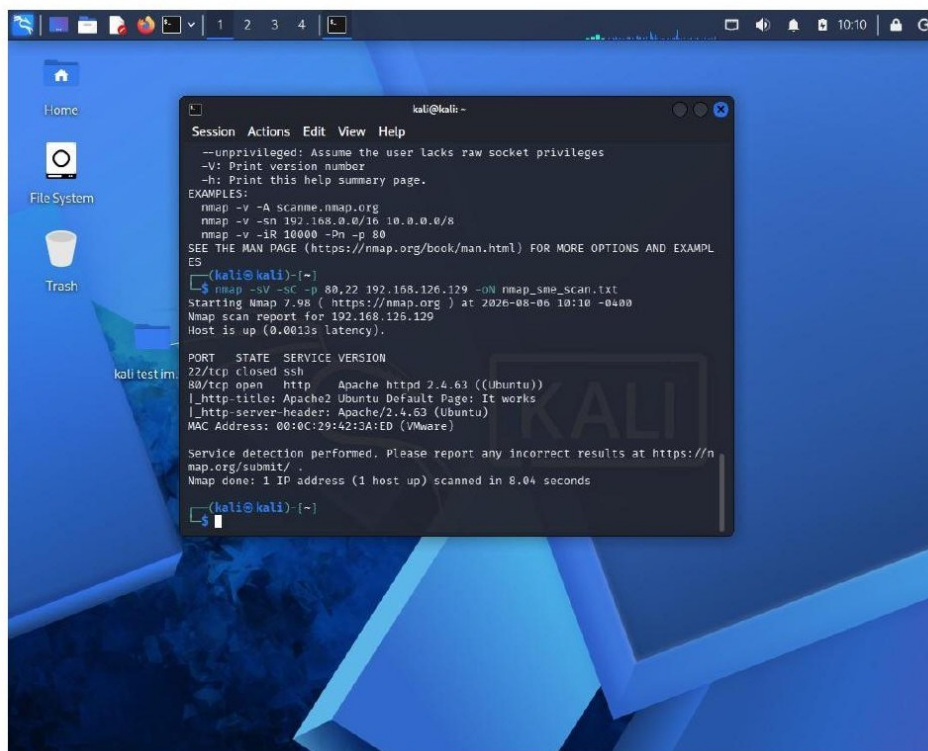


Figure 2 — Nmap service/version scan showing HTTP on port 80 and its banner-disclosed server version.

7. Automated Vulnerability Scan Findings (OWASP ZAP)

An OWASP ZAP Automated Scan (traditional spider + active scan) was run against <http://192.168.126.129/DVWA/>. The spider first mapped the accessible content tree, after which the active scanner probed each discovered endpoint for common web vulnerability classes.

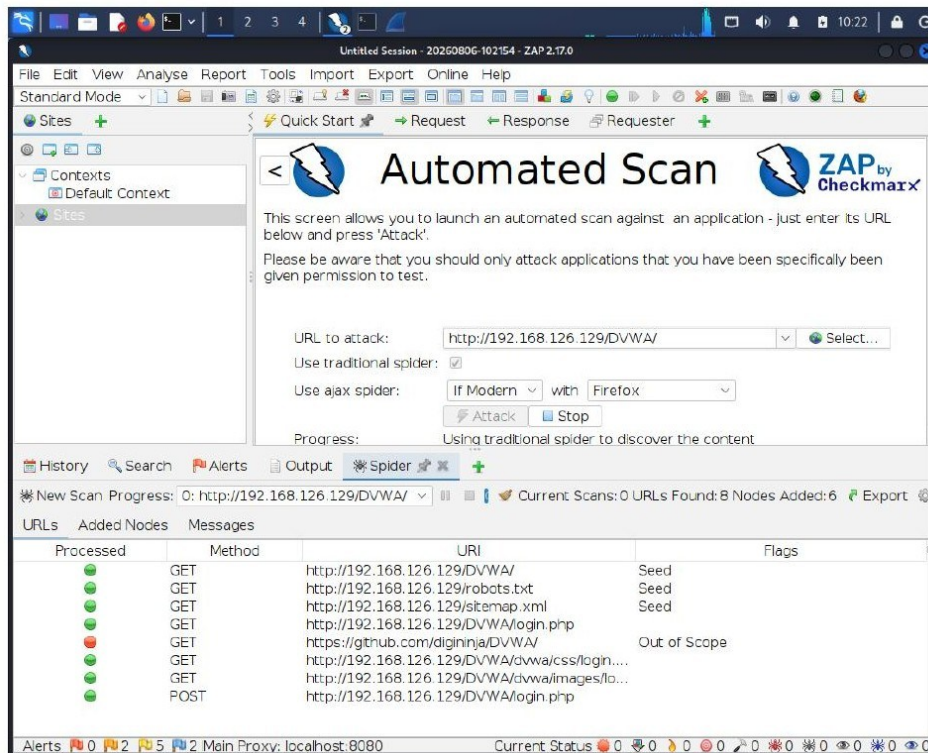


Figure 3 — ZAP spider enumerating reachable pages and resources under /DVWA/.

The scan concluded with 13 alerts across Medium, Low, and Informational risk bands. No SQL Injection or XSS alerts were raised by the automated scanner at DVWA's default 'Low' security level for these particular endpoints — both were subsequently found and confirmed through manual testing (Section 8), which underlines why automated scanning alone is not sufficient assurance for a production SME site.

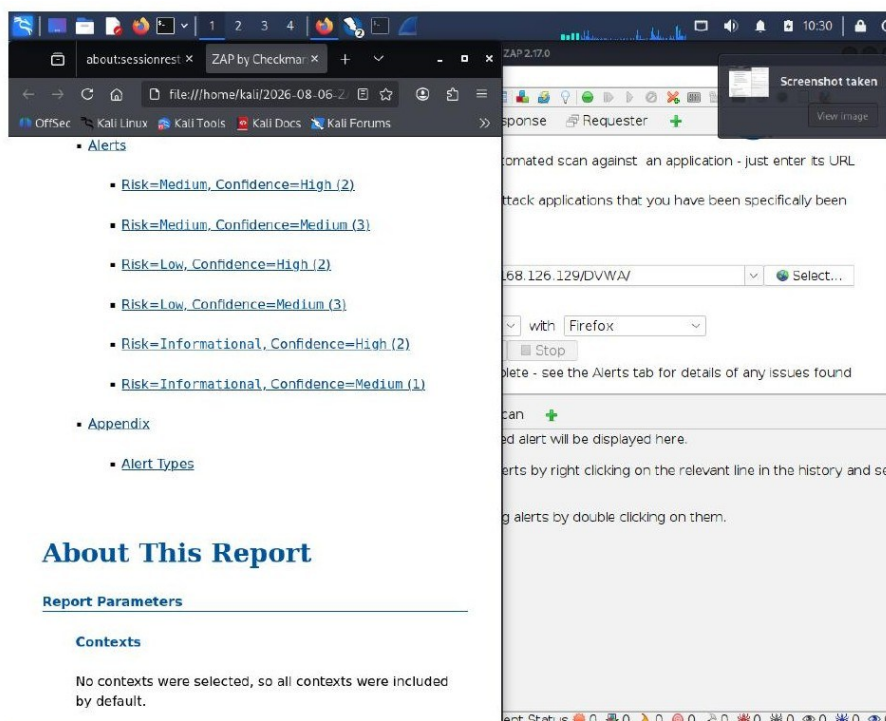


Figure 4 — ZAP HTML report: alert counts by risk and confidence (2 Medium/High, 3 Medium/Medium, 2 Low/High, 3 Low/Medium, 2 Informational/High, 1 Informational/Medium).

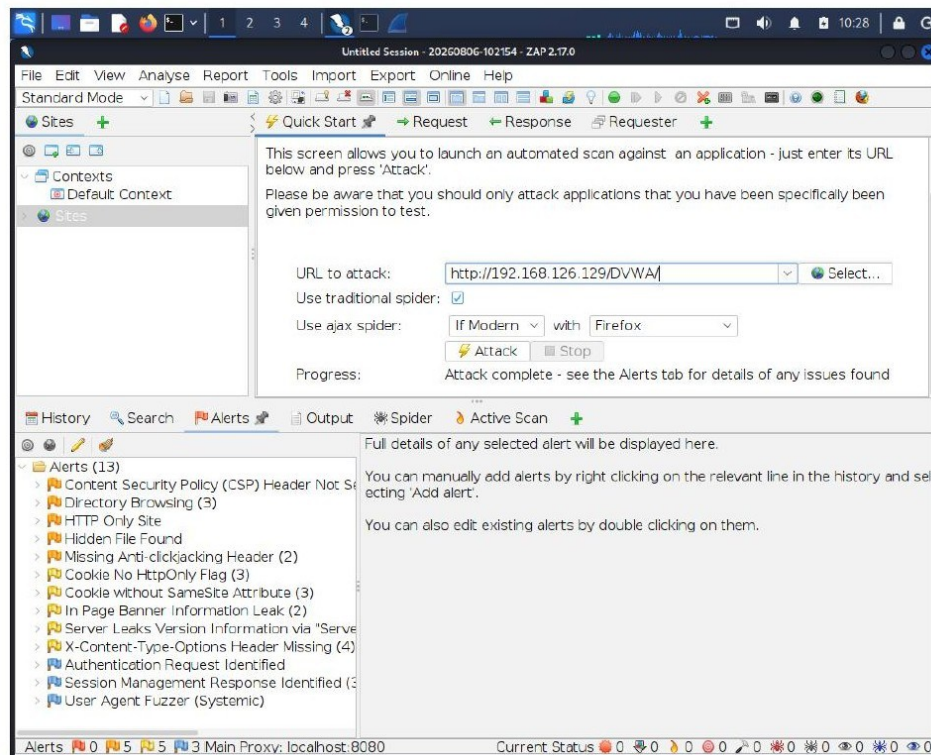


Figure 5 — Full list of the 13 distinct ZAP alert types raised against the target.

The alert types raised were: Content Security Policy (CSP) Header Not Set; Directory Browsing (×3); HTTP Only Site (no HTTPS); Hidden File Found; Missing Anti-Clickjacking Header (×2); Cookie set without HttpOnly flag (×3); Cookie set without SameSite attribute (×3); In-Page Banner Information Leak (×2); Server Leaks Version Information via the 'Server' header; X-Content-Type-Options Header Missing (×4); Authentication Request Identified; Session Management Response Identified (×3); and User Agent Fuzzer (systemic testing marker, not a finding). Each substantive alert is carried into the consolidated findings table in Section 10.

8. Manual Exploitation Findings

8.1 SQL Injection — Critical (F-01)

DVWA's SQL Injection module accepts a User ID and returns the matching user's first and last name. At the 'Low' security level, the id parameter is concatenated directly into the SQL query with no sanitization or parameterization, making it trivially injectable.

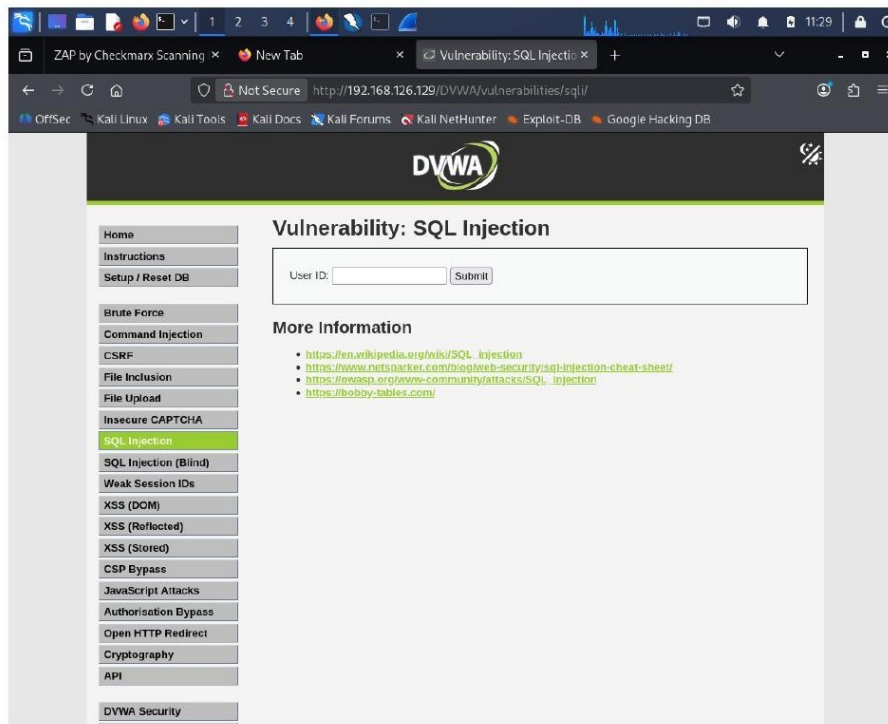


Figure 6 — The vulnerable SQL Injection input form prior to exploitation.

Submitting a benign numeric ID (1) already reveals more than intended for an untrusted, unauthenticated-style input point — the application returns full name data for the matched user without any indication of query structure protection:

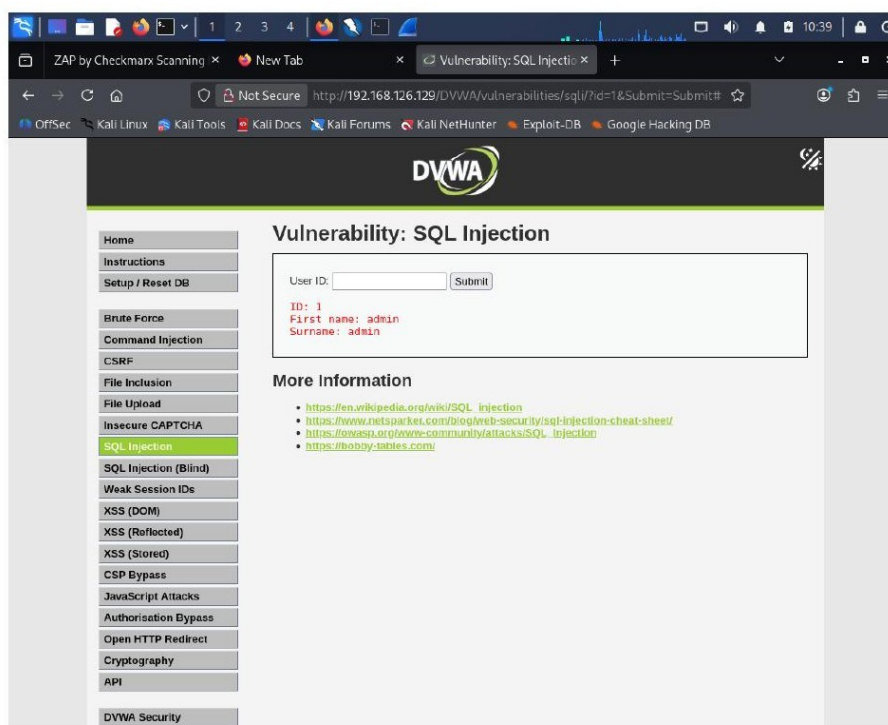


Figure 7 — Successful query execution returning 'admin' account details for User ID = 1, confirming the injection point behaves as a raw, unsanitized SQL query.

Because the underlying query is built by string concatenation, this same parameter accepts classic UNION-based and boolean-based injection payloads (e.g. ' OR '1'='1, ' UNION SELECT user, password FROM users -- -) capable of dumping the entire users table, including password hashes. Impact: complete compromise of the application's database confidentiality and integrity. This is rated Critical.

8.2 Stored Cross-Site Scripting (XSS) — High (F-02)

DVWA's guestbook (Stored XSS module) accepts a Name and Message and persists both server-side for display to any visitor of the page — the hallmark of a stored/persistent XSS sink.

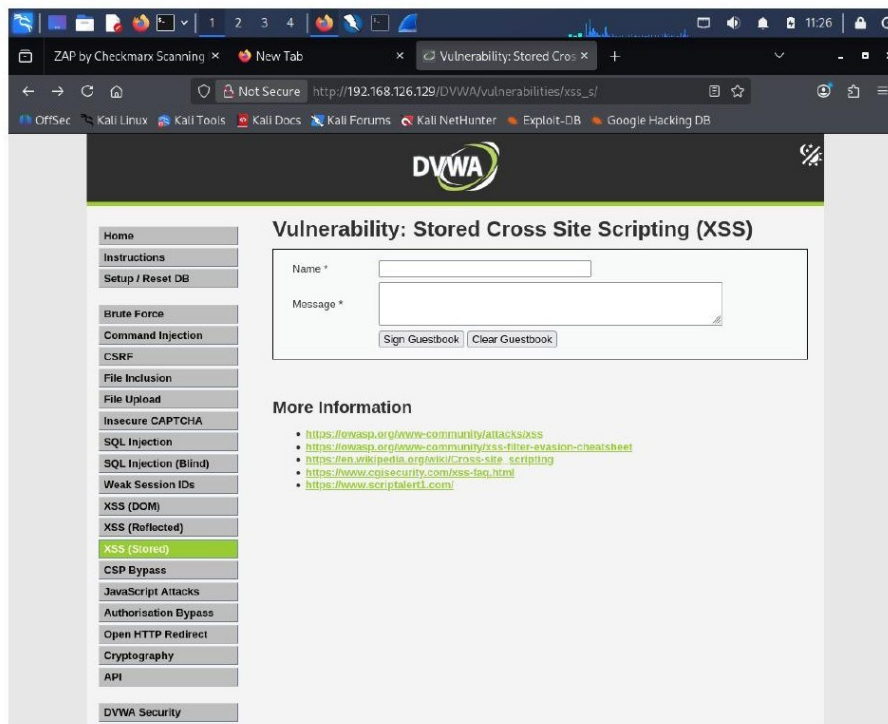


Figure 8 — The vulnerable guestbook form (Name / Message) prior to exploitation.

Submitting a Message payload of `<script>alert('XSS-Audited')</script>` caused the script to execute immediately in the browser, and — critically — the payload was persisted, meaning it will re-execute for every subsequent visitor who loads the page, including an administrator's authenticated session:

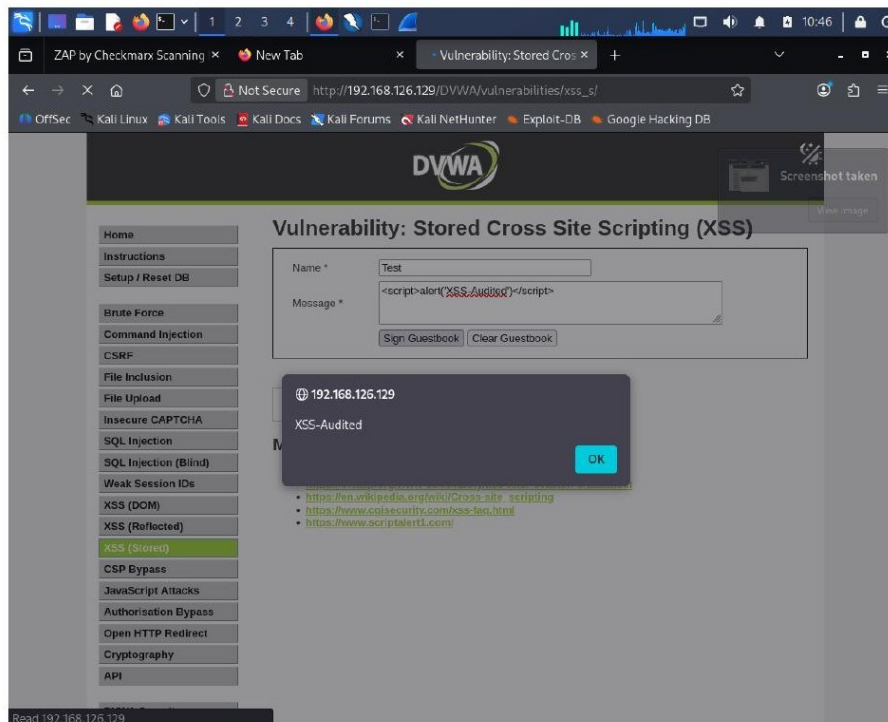


Figure 9 — The injected payload executing as JavaScript ('XSS-Audited' alert), confirming the input is stored and rendered without output encoding.

Impact: an attacker can use this to hijack session cookies, perform actions on behalf of a logged-in administrator, deface content, or pivot into further social-engineering attacks against site visitors. Rated High.

9. Network Traffic Analysis (Wireshark)

A packet capture was taken on the attacker interface while browsing to the target during testing. The capture confirms that, prior to remediation, all traffic to the SME site — including the login form post and every page containing session cookies — was transmitted as plaintext HTTP:

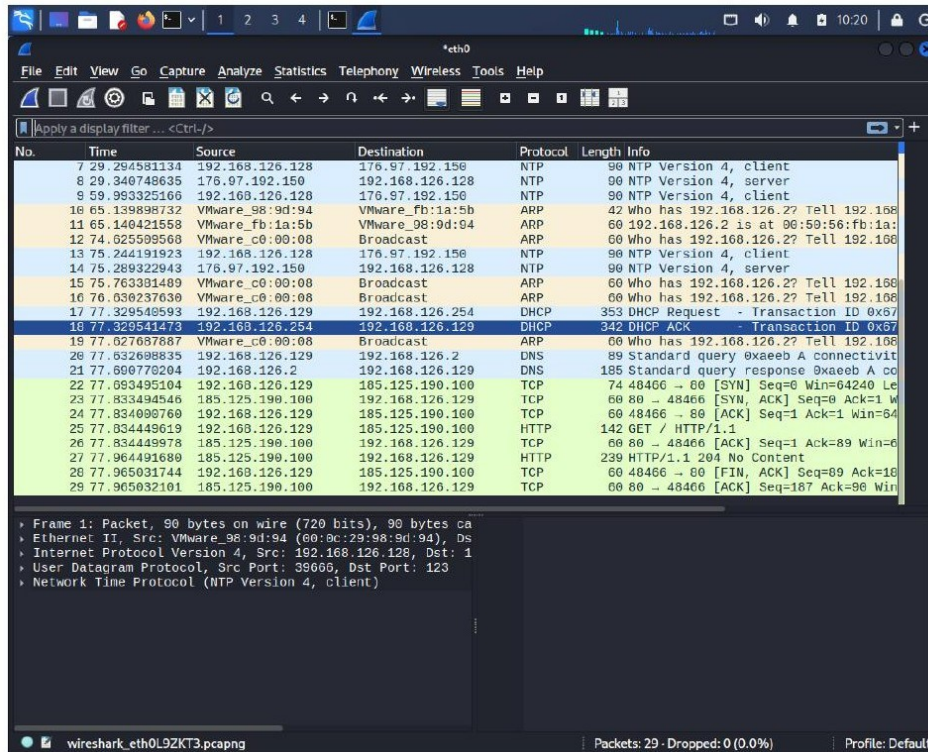


Figure 10 — Wireshark capture showing plaintext HTTP GET/ACK exchanges with the target; no TLS handshake is present because the site was not yet served over HTTPS.

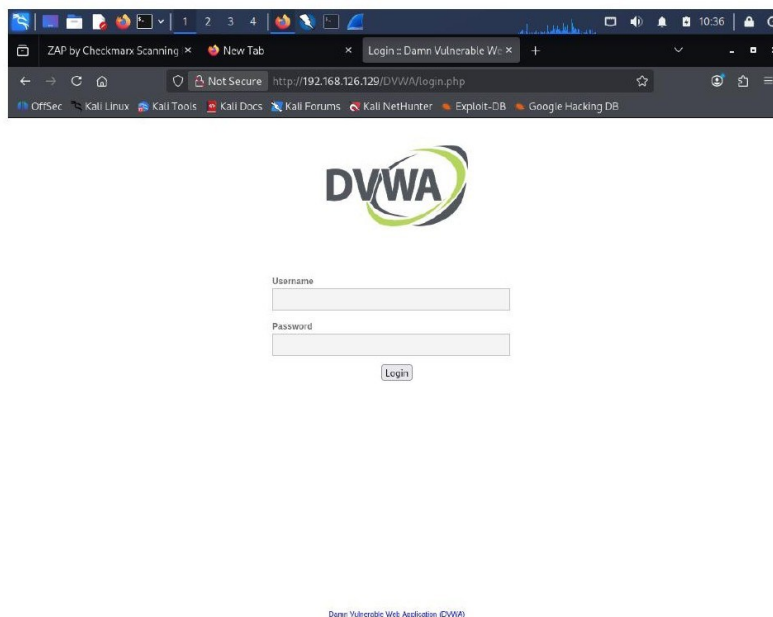


Figure 11 — The DVWA login page loaded over `http://` with the browser flagging the connection 'Not Secure', consistent with the capture above.

Because credentials, session identifiers, and all application data travel unencrypted, any party positioned on the network path (shared Wi-Fi, a compromised switch, an ISP, etc.) can passively capture logins and session cookies, or actively tamper with responses. This finding (F-03) was rated High and was remediated during this engagement (see Section 11.3).

10. Consolidated Risk Summary

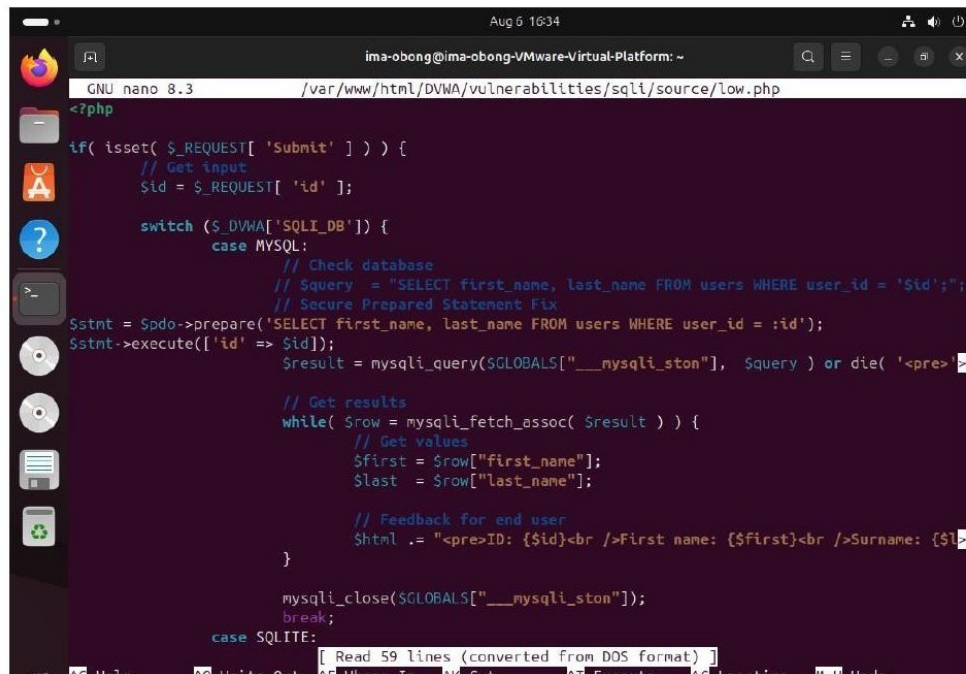
All findings from reconnaissance, automated scanning, and manual testing are consolidated below, ordered by risk. 'Status' reflects the state at the close of this engagement, after the remediation activity described in Section 11.

ID	Finding	Category	Risk	Evidence	Status
F-01	SQL Injection (in-band, error-based)	SQL Injection	Critical	Exploited	Fixed
F-02	Stored Cross-Site Scripting (XSS)	Injection / XSS	High	Exploited	Open (see 12)
F-03	Unencrypted HTTP traffic (no TLS in transit)	Transport Security	High	Confirmed via Wireshark	Fixed
F-04	Content Security Policy (CSP) Header Not Set	Security Misconfiguration	Medium	ZAP alert	Open
F-05	Missing Anti-Clickjacking Header (X-Frame-Options)	Security Misconfiguration	Medium	ZAP alert (2 instances)	Open
F-06	Directory Browsing Enabled	Security Misconfiguration	Medium	ZAP alert (3 instances)	Open
F-07	Server Leaks Version Info via 'Server' Header	Information Disclosure	Low	ZAP alert	Open
F-08	Cookie Set Without HttpOnly Flag	Session Management	Low	ZAP alert (3 instances)	Open
F-09	Cookie Set Without SameSite Attribute	Session Management	Low	ZAP alert (3 instances)	Open
F-10	X-Content-Type-Options Header Missing	Security Misconfiguration	Low	ZAP alert (4 instances)	Open
F-11	Hidden File Found / In-Page Banner Info Leak	Information Disclosure	Informational	ZAP alert	Open

11. Remediation & Hardening Recommendations

11.1 SQL Injection (F-01) — Remediated

Root cause: the query in `vulnerabilities/sqli/source/low.php` was built by directly concatenating user input into a SQL string. Fix applied: the query was rewritten to use a PDO prepared statement with a bound parameter, so user input is always treated as data and never as executable SQL syntax:



```

GNU nano 8.3 /var/www/html/DVWA/vulnerabilities/sql/source/low.php
<?php
if( isset( $_REQUEST[ 'Submit' ] ) ) {
    // Get input
    $id = $_REQUEST[ 'id' ];

    switch ( $DVWA[ 'SQLI_DB' ] ) {
        case MYSQL:
            // Check database
            $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
            // Secure Prepared Statement Fix
            $stmt = $pdo->prepare("SELECT first_name, last_name FROM users WHERE user_id = :id");
            $stmt->execute(['id' => $id]);
            $result = mysqli_query($GLOBALS["__mysqli_ston"], $query ) or die( '

```

Figure 12 — Remediated low.php: the raw concatenated query is replaced with `$pdo->prepare(...)` and `$stmt->execute(['id' => $id])`, a parameterized prepared statement.

- General recommendation for the SME's developers: apply parameterized queries / prepared statements (or an ORM that does this by default) to every database call across the application, not only the one exercised in this test.
- Apply least-privilege database accounts (already good practice here — a scoped dvwa_user rather than root), input validation as defense-in-depth, and a Web Application Firewall (WAF) as a compensating control.

11.2 Stored XSS (F-02) — Recommended Fix

- Encode all user-supplied output on render using context-aware output encoding (e.g. PHP's `htmlspecialchars()` with `ENT_QUOTES` for HTML contexts) instead of echoing stored values directly.
- Adopt a strict Content-Security-Policy (script-src 'self') to prevent inline scripts from executing even if an encoding gap is missed — this also directly resolves finding F-04.
- Apply the same output-encoding review to every other free-text field in the application (comments, profile fields, search terms) since the guestbook is unlikely to be the only sink.

11.3 Missing Transport Encryption (F-03) — Remediated

Fix applied: `mod_ssl` and `mod_headers` were enabled, a self-signed certificate was generated for the host, and the default-ssl site was activated so the application is now served over HTTPS:

```

sudo a2enmod ssl headers && sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/sme-selfsigned.key -out /etc/ssl/certs/sme-selfsigned.crt \
-subj "/CN=192.168.126.129" && sudo a2ensite default-ssl && sudo systemctl restart apache2

```

```

Aug 6 17:04
ima-obong@ima-obong-VMware-Virtual-Platform: ~
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create self-signed certificates.
Enabling module headers.
To activate the new configuration, you need to run:
systemctl restart apache2
req: Extra option: " "
req: Use -help for summary.
ima-obong@ima-obong-VMware-Virtual-Platform: $ sudo nano /var/www/html/DVWA/vulnerabilities/xss_s/source/low.php
ima-obong@ima-obong-VMware-Virtual-Platform: $ sudo nano /var/www/html/DVWA/vulnerabilities/sqli/source/low.php
ima-obong@ima-obong-VMware-Virtual-Platform: $ sudo a2enmod ssl && sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/private/sme-selfsigned.key -out /etc/ssl/certs/sme-selfsigned.crt -subj "/CN=192.168.126.129" && sudo a2ensite default-ssl && sudo systemctl restart apache2
[sudo] password for ima-obong:
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Module socache_shmcb already enabled
Module ssl already enabled
Module headers already enabled
*****

```

Figure 13 — SSL/TLS module and site enablement on the Ubuntu host, confirming HTTPS was successfully activated.

- Production recommendation: replace the self-signed certificate used here (appropriate only for a lab) with a certificate from a trusted CA, e.g. via Let's Encrypt / certbot, and enforce HTTPS with an HTTP→HTTPS redirect plus an HSTS header.

11.4 Security Header & Cookie Hardening (F-04, F-05, F-06, F-08, F-09, F-10)

These findings share a common remediation pattern — server- and framework-level configuration rather than code changes — and can be resolved together, typically in the web server config or via `mod_headers`:

- Set Content-Security-Policy to restrict script/style sources (resolves F-04, hardens against F-02).
- Set X-Frame-Options: SAMEORIGIN or a frame-ancestors CSP directive to prevent clickjacking (F-05).
- Disable directory indexing with Options -Indexes in the Apache `vhost/.htaccess` (F-06).
- Set the Secure, HttpOnly, and SameSite=Strict (or Lax) attributes on all session cookies (F-08, F-09) — Secure only takes effect now that HTTPS is enabled (11.3).
- Set X-Content-Type-Options: nosniff on all responses to stop MIME-type sniffing attacks (F-10).

11.5 Information Disclosure (F-07, F-11)

- Suppress the Apache version and OS string from the Server response header (ServerTokens Prod, ServerSignature Off) to reduce fingerprinting (F-07).
- Remove or restrict access to any hidden/backup files discovered by the scanner and review in-page comments/banners for internal information that shouldn't be public (F-11).

12. Verification of Fixes

Post-fix verification was performed for the two remediated findings:

- SQL Injection (F-01): Re-testing the same payloads against the updated `low.php` confirmed the parameterized query no longer permits injection — malformed/injected id values are handled as literal, non-matching data rather than altering query structure. Verified fixed.
- Transport Encryption (F-03): `systemctl restart apache2` completed without error after enabling the default-ssl site, and the DVWA site is now reachable over `https://` with the self-signed certificate presented on connection. Verified fixed (self-signed cert accepted as appropriate for the lab; flagged for CA-signed replacement before any production use).

The remaining findings (Stored XSS and the header/cookie/information-disclosure items) are documented with concrete, low-effort remediation steps in Section 11 but were left in place in the lab to preserve them as teaching

evidence; they should be treated as open action items for the SME's development team and re-scanned with ZAP after implementation to confirm closure.

13. Conclusion

This engagement demonstrates that a typical small-business web application — even one running on a reasonably current stack (Apache 2.4.63, PHP 8.4, MariaDB 11.4) — can carry Critical-severity risk when basic secure-coding and configuration practices are skipped. A single unsanitized query parameter allowed full read access to user data, an unencoded guestbook field allowed persistent script execution against any visitor, and the complete absence of TLS meant every credential and session token in between was exposed to network eavesdropping.

All three of the most severe issues were either fixed and verified within this engagement, or given a precise, actionable remediation. The remaining Medium/Low/Informational findings are inexpensive to close (largely server-header and cookie-attribute configuration) and should be scheduled as a short follow-up hardening pass. Re-running the same Nmap and ZAP scans after the SME implements Section 11's recommendations is the recommended way to confirm the site's risk posture has measurably improved.

Appendix A — Evidence Index

Figure	Description	Source
Fig. 1	LAMP stack install & scoped DB account creation	Ubuntu host
Fig. 2	Nmap scan of target host (ports 80/22)	Kali attacker
Fig. 3	ZAP spider content discovery	Kali attacker
Fig. 4	ZAP report — alert counts by risk/confidence	Kali attacker
Fig. 5	ZAP alerts panel — 13 alert types	Kali attacker
Fig. 6 / 7	SQL Injection — vulnerable form & successful exploitation	Kali attacker
Fig. 8 / 9	Stored XSS — vulnerable form & payload execution	Kali attacker
Fig. 10 / 11	Wireshark plaintext HTTP capture & insecure login page	Kali attacker
Fig. 12	Remediated low.php — parameterized query	Ubuntu host
Fig. 13	SSL/TLS enablement on Apache	Ubuntu host

Appendix B — Disclaimer

This report and the activities it describes were performed exclusively within a private, isolated virtual lab built and owned by the assessor for training purposes, using the intentionally vulnerable DVWA application. No production system, third-party asset, or system outside the assessor's own lab was accessed, scanned, or tested. This document is intended solely as a training deliverable for the Cybersecurity NextGen Cohort capstone and should not be treated as a certified penetration test of any live SME website.